



PRACTICAL 2

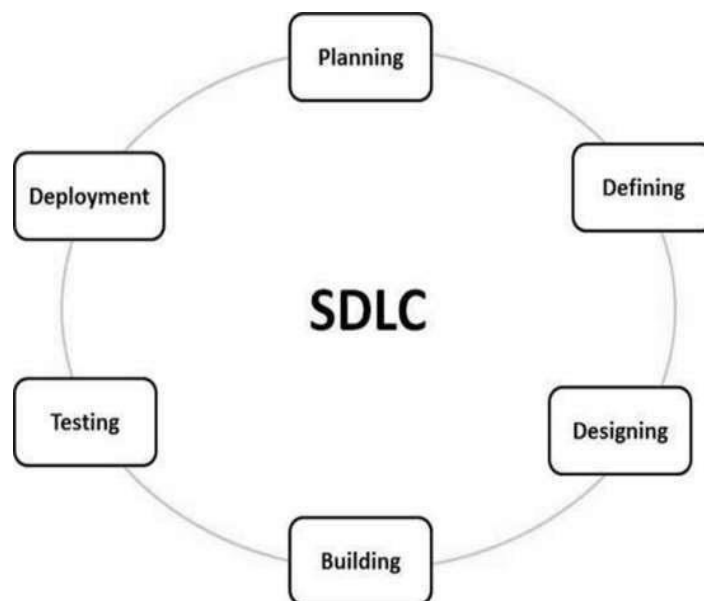
Aim:

Identify Suitable Design and Implement model from the different software engineering models.

Software Process Model:

- The process model is the abstract representation of process.
- Also known as Software development life cycle (SDLC) or Application development life cycle Models
- Process models prescribe a distinct set of activities, actions, tasks and milestones (deliverables) required to engineer high quality software.
- Process models are not perfect, but provide roadmap for software engineering work.
- Software models provide stability, control and organization to a process that if not managed can easily get out of control.
- Software process models are adapted (adjusted) to meet the needs of software engineers and managers for a specific project.

SDLC Phase





Stage 1: Planning and Requirement Analysis

Requirement analysis is the most important and fundamental stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility study in the economical, operational and technical areas.

Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage. The outcome of the technical feasibility study is to define the various technical approaches that can be followed to implement the project successfully with minimum risks.

Stage 2: Defining Requirements

Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts. This is done through an **SRS (Software Requirement Specification)** document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a DDS - Design Document Specification.

This DDS is reviewed by all the important stakeholders and based on various parameters as risk assessment, product robustness, design modularity, budget and time constraints, the best design approach is selected for the product.

A design approach clearly defines all the architectural modules of the product along with its communication and data flow representation with the external and third party modules (if any). The internal design of all the modules of the proposed architecture should be clearly defined with the minutest of the details in DDS.

Stage 4: Building or Developing the Product

In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.



Developers must follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers, etc. are used to generate the code. Different high level programming languages such as C, C++, Pascal, Java and PHP are used for coding. The programming language is chosen with respect to the type of software being developed.

Stage 5: Testing the Product

This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC. However, this stage refers to the testing only stage of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

Once the product is tested and ready to be deployed it is released formally in the appropriate market. Sometimes product deployment happens in stages as per the business strategy of that organization. The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).

Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment. After the product is released in the market, its maintenance is done for the existing customer base.

List of Models:

- Linear Sequential Waterfall model
- Iterative Model
- Prototype Model
- Incremental Model
- Evolutionary Software Process Model
- Spiral Model
- V Model
- RAD Model.

Difference between Models:



Parul™
University

Faculty of Engineering & Technology
Subject Name : Software Engineering
Subject Code : 303105254
B.Tech CSE-AI Year 2025-26 Semester 4th

Properties of Model	Water-Fall Model	Incremental Model	Spiral Model	Rad Model
Planning in early stage	Yes	Yes	Yes	No
Returning to an earlier phase	No	Yes	Yes	Yes
Handle Large-Project	Not Appropriate	Not Appropriate	Appropriate	Not Appropriate
Detailed Documentation	Necessary	Yes but not much	Yes	Limited
Cost	Low	Low	Expensive	Low
Requirement Specifications	Beginning	Beginning	Beginning	Time boxed release
Flexibility to change	Difficult	Easy	Easy	Easy
User Involvement	Only at beginning	Intermediate	High	Only at the beginning
Maintenance	Least	Promotes Maintainability	Typical	Easily Maintained
Duration	Long	Very long	Long	Short
Risk Involvement	High	Low	Medium to high risk	Low
Framework Type	Linear	Linear + Iterative	Linear + Iterative	Linear
Testing	After completion of coding phase	After every iteration	At the end of the engineering phase	After completion of coding
Overlapping Phases	No	Yes (As parallel development is there)	No	Yes
Maintenance	Least Maintainable	Maintainable	Yes	Easily Maintainable
Re-usability	Least possible	To some extent	To some extent	Yes
Time-Frame	Very Long	Long	Long	Short
Working software availability	At the end of the life-cycle	At the end of every iteration	At the end of every iteration	At the end of the life cycle
Objective	High Assurance	Rapid Development	High Assurance	Rapid development
Team size	Large Team	Not Large Team	Large Team	Small Team
Customer control over administrator	Very Low	Yes	Yes	Yes



Diagram For Selected Model and It Description:-

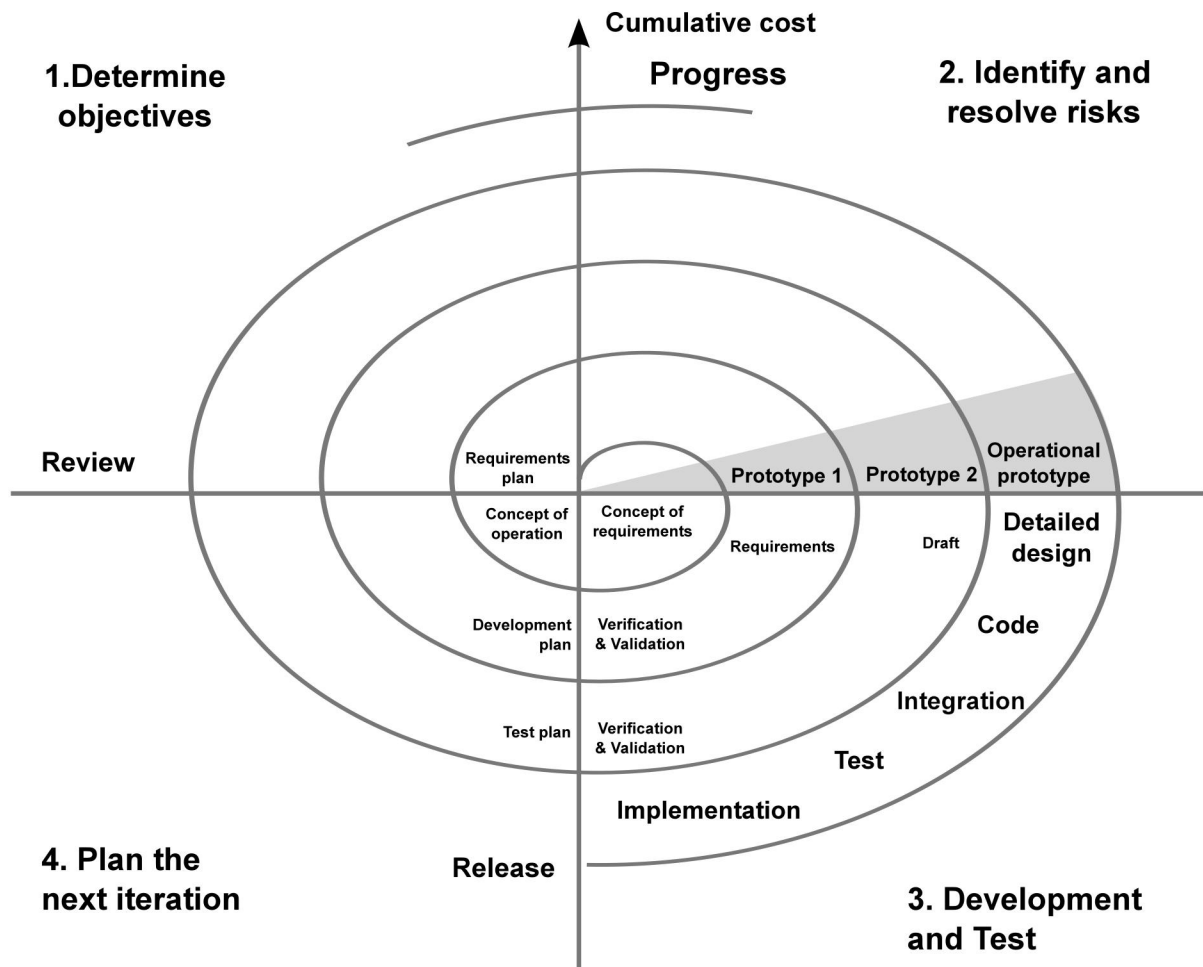
Spiral Model:

- **Cyclic Nature:** Unlike a straight line, the project moves in "spirals," where each loop represents a complete phase of the software process (planning, risk analysis, engineering, and evaluation).
- **Risk-Centric:** It is the only model that mandates a formal **Risk Analysis** in every iteration, making it ideal for high-stakes projects like financial or medical software.
- **Evolutionary Progress:** The "spiral" gets wider as the project progresses, signifying that the software becomes more complete and the total cost/time increases with each cycle.
- **Client Involvement:** The user or client evaluates the product at the end of every loop, ensuring the final trading system aligns perfectly with user expectations.

● When to use?

- **High Risk:** The project has significant technical or financial stakes (like your trading system).
- **Large Scale:** The system is complex and requires long-term development.
- **Vague Requirements:** You need to build prototypes to figure out exactly what the user wants.
- **Need for Flexibility:** You expect to make major changes or add new features during the development process.
- **Quality is Priority:** Stability and security are more important than finishing the project as quickly as possible.

Diagram of Spiral Model:



Phases of Spiral Model:-

1. Planning Phase

- Identify project objectives and system requirements.
- Define project scope, resources, cost, and schedule.
- Analyze possible alternatives for system development.

2. Risk Analysis Phase

- Identify potential risks related to security, performance, cost, and technology.
- Evaluate and prioritize identified risks.
- Develop prototypes or solutions to reduce risks.

3. Engineering Phase

- Design the system architecture and components.
- Perform coding and unit testing of modules.
- Develop the system incrementally in iterations.

4. Evaluation Phase

- Review the developed system with users and stakeholders.
- Collect feedback and validate requirements.
- Decide whether to proceed to the next iteration.

Advantages of Spiral Model:-

- Focuses on early identification and reduction of risks.
- Allows continuous user feedback and requirement changes.
- Supports incremental development and testing.
- Suitable for large, complex, and high-risk systems.
- Improves software quality through repeated evaluation.

Disadvantages of Spiral Model:-

- Complex and difficult to manage.
- Requires experienced developers and project managers.
- Cost of development is high due to repeated iterations.
- Not suitable for small or low-risk projects.
- Risk analysis phase can be time-consuming.

Justification: That why you choose this model only:-

- **Continuous Risk Assessment:** Trading systems are high-risk due to financial transactions and data sensitivity. The Spiral Model's primary strength is identifying and resolving risks (like latency issues or security gaps) at every stage of the cycle.
- **Modular Feature Rollout:** You can start with a core engine (market data) in the first spiral and progressively add complex features like "limit orders," "margin trading," or "portfolio analytics" in later loops.
- **Prototyping for UI/UX:** Traders need a highly responsive interface. The model allows for early prototyping, enabling you to gather feedback on the dashboard layout before the final backend is even completed.
- **Regulatory Compliance:** Financial laws change frequently. The iterative nature of the Spiral Model allows you to adapt to new legal requirements or tax calculations without having to restart the entire development process.
- **System Stability:** Since testing and evaluation happen at the end of *every* loop, the final product is significantly more stable and "battle-tested" than a system built using a linear approach.
- **High Customization:** It allows for the integration of third-party APIs (like banking gateways or stock exchange feeds) to be tested individually in separate iterations to ensure they don't crash the core system.